

Encontre a Ordem

- **Limite de tempo:** 10 minutos
- **Ambiente:** uma GPU (≈ 16 GB VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Armazenamento:** 5 GB

Problema

São fornecidos diálogos falados em inglês entre dois participantes, *Falante A* e *Falante B*. Cada diálogo é segmentado em turnos de fala, com cada turno contendo a fala de apenas um falante. Cada turno é armazenado como um arquivo de áudio `.wav` separado, de modo que um diálogo completo é representado por um conjunto de arquivos `.wav`, um para cada turno.

Infelizmente, os turnos foram embaralhados aleatoriamente, portanto a conversa não faz mais sentido. No nome de arquivo `chunk_{k}.wav`, k refere-se ao k -ésimo trecho no conjunto embaralhado, não ao k -ésimo turno no diálogo original.

!! Sua tarefa é reconstruir a ordem cronológica original da conversa.



Dataset

Cada diálogo contém arquivos de áudio `n` denominados `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. Os trechos são turnos individuais. Os nomes dos arquivos correspondem apenas à ordem embaralhada. Eles não indicam onde um trecho pertence na conversa original. Cada diálogo tem 7-20 trechos, mono, 44.1 kHz (você pode reamostrar).

`prefix.json` contém os índices dos nomes dos arquivos dos dois primeiros trechos de cada diálogo. Isso identifica o verdadeiro início do diálogo e elimina a ambiguidade entre ler a conversa para a frente ou para trás.

Por exemplo: 11: [7, 12] significa que o primeiro e o segundo turnos do diálogo 11 são `chunk_7.wav` e `chunk_12.wav`, respectivamente.

O que você recebe

Você recebe **duas pastas em formato idêntico:**

Pasta	Diálogos	<code>answers.json</code> ?	Use-a para
<code>dataset/train/</code>	1,288	☐ incluído	treinar / fazer ajuste fino do seu modelo
<code>dataset/test_public/</code>	100	☐ incluído	executar seu pipeline e calcular localmente sua própria pontuação

Durante a avaliação, sua pasta `dataset/test_public/` é substituída de forma transparente por uma `hidden evaluation set` (`test_leaderboard_a` para o placar público e `test_leaderboard_b` para o placar final) — elas têm o mesmo tamanho e formato que `dataset/test_public/`, mas sem `answers.json`.

Seu notebook é executado novamente nesses dados, e o arquivo `answers.json` que ele produz é usado para a pontuação. Os diálogos de teste reservados vêm da mesma distribuição que `train`, portanto sua pontuação local de `test_public` é uma prévia fiel.

Estrutura de diretórios

```
dataset/train/
  prefix.json  # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P}  ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json      # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Saída

Para cada diálogo, determine a ordem cronológica original de seus trechos de áudio. Sua previsão deve ser uma permutação P de $\{0, 1, \dots, n-1\}$, em que $P[i]$ é a posição cronológica prevista de `chunk_i.wav` ($0 = \text{primeiro}$).

Seu arquivo de saída `answers.json` deve mapear cada ID de diálogo para sua permutação prevista:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Exemplo

Um diálogo tem 3 trechos embaralhados `chunk_0`, `chunk_1`, `chunk_2`:

trecho embaralhado	conteúdo falado	posição verdadeira (posição no ranking)
chunk_0.wav	"No worries — I'll send you the notes afterwards."	2 (último)
chunk_1.wav	"Hey, are you coming to the three o'clock meeting?"	0 (primeiro)
chunk_2.wav	"I can't — I've got a dentist appointment then."	1

A ordem verdadeira é **chunk_1** → **chunk_2** → **chunk_0**, portanto $P = [2, 0, 1]$, e `prefix.json` contém `[1, 2]`.

⚠ **P deve ser uma permutação genuína:** comprimento n , indexada a partir de 0, com cada valor exatamente uma vez. Valores duplicados, ausentes ou fora do intervalo (por exemplo, indexados a partir de 1) resultam em pontuação 0 para esse diálogo, assim como um diálogo ausente do arquivo. Um arquivo malformatado ou que não seja JSON é rejeitado.

Pontuação

A métrica de pontuação desta tarefa é a **acurácia de ordenação par a par**. Ela verifica cada par de trechos e pergunta: *qual dos dois deve vir primeiro?* Um par está correto se sua previsão der a mesma resposta que a referência verdadeira. Para um diálogo com n trechos, há

$$M = n(n - 1)/2$$

pares; seja I o número de inversões — pares ordenados de forma diferente da referência verdadeira:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

! A pontuação final é a média das pontuações por diálogo sobre todos os diálogos na divisão.

Modelos permitidos

Você só pode usar os seguintes modelos pré-treinados para resolver esta tarefa, tanto durante o treinamento quanto durante a avaliação. Todos esses modelos já estão baixados e disponíveis no ambiente. Você pode ver exemplos de como usá-los no notebook `baseline solution.ipynb`. Observe que você não pode usar nenhum outro modelo e que seu programa não tem acesso à internet.

- **Representações de fala: wav2vec 2.0.** O **encoder do Whisper** também pode ser usado como extrator de características. [Documentação do modelo wav2vec](#)

- **Reconhecimento automático de fala (ASR): OpenAI Whisper** (qualquer tamanho). [Documentação do modelo Whisper](#)
- **Modelo de linguagem: Qwen2.5-0.5B**, que pode ser usado em zero-shot ou submetido a ajuste fino na divisão `train` fornecida. [Documentação do modelo Qwen](#) Observe que o limite de 10 minutos deve abranger qualquer treinamento ou ajuste fino que você realizar durante a avaliação, além da inferência no conjunto de avaliação.

Como enviar

- Abra `solution.ipynb` e execute todas as células. Confirme que ele grava `answers.json` no diretório de trabalho com uma permutação para cada diálogo em `dataset/test_public/` (100 diálogos). Durante a avaliação, o notebook é executado novamente no conjunto de teste oculto, e o `answers.json` produzido nele recebe a pontuação.
- Melhore a solução se quiser — ou não; o baseline por si só valida o pipeline.
- Abra a aba Git na barra lateral esquerda do JupyterLab.
- **Adicione à área de preparação (Stage)** `solution.ipynb` (o ícone + ao lado dele).
- Insira uma mensagem de commit e clique em **Commit**.
- Clique no ícone de nuvem com uma seta para cima para fazer push.
- Retorne a esta página da Competição e clique em **Submit**.

Envie exatamente um arquivo, denominado `solution.ipynb`.